Task 7: Interactive Web Form with Events & Functions

An interactive web form uses **JavaScript events** to:

- Validate input
- Improve user experience
- Provide instant feedback

Features Required

✔ Validate inputs on keypress
✔ Highlight fields on mouse hover
✔ Show confirmation on double-click submit
✔ Use reusable JavaScript functions

Code

HTML + JavaScript

```
<!DOCTYPE html>
<html>
<head>
    <title>Expense Feedback Form</title>
    <style>
        body {
            font-family: Arial;
            padding: 20px;
        }

        input, textarea, select {
            padding: 8px;
            margin: 8px 0;
            width: 250px;
        }

        .highlight {
            border: 2px solid blue;
        }

        .error {
            border: 2px solid red;
        }
    </style>

    <script>
        // Validate Name (letters only)
```

```
        function validateName(event) {
            let char = event.key;
            if (!/^[a-zA-Z ]$/.test(char)) {
                event.preventDefault();
            }
        }

        // Validate Amount (numbers only)
        function validateAmount(event) {
            let char = event.key;
            if (!/^[0-9]$/.test(char)) {
                event.preventDefault();
            }
        }

        // Highlight field
        function highlightField(element) {
            element.classList.add("highlight");
        }

        function removeHighlight(element) {
            element.classList.remove("highlight");
        }

        // Double click submit
        function submitForm() {
            let name = document.getElementById("name").value;
            let amount = document.getElementById("amount").value;

            if (name === "" || amount === "") {
                alert("Please fill all required fields!");
                return;
            }

            alert("Expense feedback submitted successfully!");
        }
    </script>
</head>

<body>

    <h2> 💰 Expense Feedback Form</h2>

    <form>
        Name: <br>
        <input type="text" id="name"
            onkeypress="validateName(event)"
            onmouseover="highlightField(this)"
            onmouseout="removeHighlight(this)">
        <br>

        Expense Category: <br>
        <select
            onmouseover="highlightField(this)"
            onmouseout="removeHighlight(this)">
```

```
        <option>Food</option>
        <option>Transport</option>
        <option>Shopping</option>
        <option>Entertainment</option>
    </select>
    <br>

    Amount (₹): <br>
    <input type="text" id="amount"
        onkeypress="validateAmount(event)"
        onmouseover="highlightField(this)"
        onmouseout="removeHighlight(this)">
    <br>

    Feedback: <br>
    <textarea
        onmouseover="highlightField(this)"
        onmouseout="removeHighlight(this)">
    </textarea>
    <br>

    <button type="button" ondblclick="submitForm()">
        Submit (Double Click)
    </button>
  </form>

</body>
</html>
```

Explanation

Keypress Validation

onkeypress="validateInput(event)"

Allows only letters
Prevents numbers/symbols

Mouse Hover Highlight

onmouseover="highlightField(this)"
onmouseout="removeHighlight(this)"

Adds border when mouse enters
Removes when mouse leaves

Double Click Submit

ondblclick="submitForm()"

Prevents accidental submission
Shows confirmation message

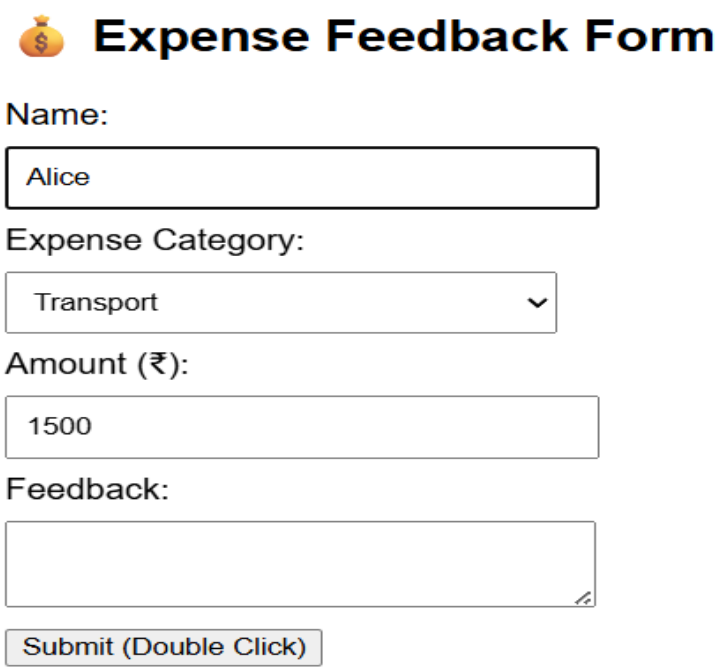
Reusable Functions

Functions used:

- `validateInput()`
- `highlightField()`
- `submitForm()`

Can be reused in multiple forms

Output :



Task 6: Automated Logging using Triggers & Views A trigger that logs every INSERT or UPDATE A view that shows daily activity reports Real-Time Usage: Audit logging in enterprise databases.

Create Table

```
CREATE TABLE employees (
  emp_id INT PRIMARY KEY,
  name VARCHAR(100),
  salary DECIMAL(10,2)
);
```

Create Audit Log Table

This table will store all INSERT and UPDATE actions.

```
CREATE TABLE employee_logs (
  log_id INT AUTO_INCREMENT PRIMARY KEY,
  emp_id INT,
  action_type VARCHAR(10),
  old_salary DECIMAL(10,2),
  new_salary DECIMAL(10,2),
  action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Create Trigger for INSERT

```
DELIMITER $$

CREATE TRIGGER after_employee_insert
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
  INSERT INTO employee_logs (emp_id, action_type, new_salary)
  VALUES (NEW.emp_id, 'INSERT', NEW.salary);
END $$

DELIMITER ;
```

Create Trigger for UPDATE

```
DELIMITER $$

CREATE TRIGGER after_employee_update
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
  INSERT INTO employee_logs (emp_id, action_type, old_salary, new_salary)
  VALUES (OLD.emp_id, 'UPDATE', OLD.salary, NEW.salary);
END $$

DELIMITER ;
```

Test Your Triggers

Insert Data

```
INSERT INTO employees VALUES (1, 'Arjun', 30000);
```

Update Data

```
UPDATE employees SET salary = 35000 WHERE emp_id = 1;
```

Check Logs Table

```
SELECT * FROM employee_logs;
```

Create View

```
CREATE VIEW daily_activity_report AS

SELECT
  DATE(action_time) AS activity_date,
  action_type,
  COUNT(*) AS total_actions,
  SUM(new_salary) AS total_salary_changes
FROM employee_logs
GROUP BY DATE(action_time), action_type;
```

View Report

SELECT * FROM daily_activity_report;

OUTPUT :

```
mysql> DELIMITER ;
mysql> INSERT INTO employees VALUES (1, 'Arjun', 300000);
Query OK, 1 row affected (0.02 sec)

mysql> UPDATE employees SET salary = 350000 WHERE emp_id = 1;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM employee_logs;
+--------+--------+-------------+------------+------------+---------------------+
| log_id | emp_id | action_type | old_salary | new_salary | action_time         |
+--------+--------+-------------+------------+------------+---------------------+
|      1 |      1 | INSERT      |       NULL |  300000.00 | 2026-04-06 22:45:08 |
|      2 |      1 | UPDATE      |  300000.00 |  350000.00 | 2026-04-06 22:45:21 |
+--------+--------+-------------+------------+------------+---------------------+
2 rows in set (0.00 sec)

mysql> CREATE VIEW daily_activity_report AS
    -> SELECT
    ->     DATE(action_time) AS activity_date,
    ->     action_type,
    ->     COUNT(*) AS total_actions,
    ->     SUM(new_salary) AS total_salary_changes
    -> FROM employee_logs
    -> GROUP BY DATE(action_time), action_type;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM daily_activity_report;
+---------------+-------------+---------------+----------------------+
| activity_date | action_type | total_actions | total_salary_changes |
+---------------+-------------+---------------+----------------------+
| 2026-04-06    | INSERT      |             1 |            300000.00 |
| 2026-04-06    | UPDATE      |             1 |            350000.00 |
+---------------+-------------+---------------+----------------------+
2 rows in set (0.02 sec)

mysql> |
```